

4.3 贪心法得不到最优解的处理

$n=1,2$ 贪心法是最优解

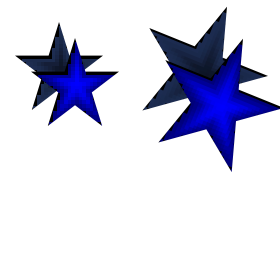
$n=1$ 只有一种零钱, $F_1(y) = G_1(y)$

$n=2$, x_2 越大, 得到的解越好

$$F_2(y) = \min_{0 \leq x_2 \leq \lfloor y/v_2 \rfloor} \{F_1(y - v_2 x_2) + w_2 x_2\}$$

$$\begin{aligned} & F_1(y - v_2(\underline{x_2 + \delta})) + w_2(\underline{x_2 + \delta}) \\ & - [F_1(y - v_2 \underline{x_2}) + w_2 \underline{x_2}] \\ = & [w_1(\underline{y - v_2 x_2 - v_2 \delta}) + \underline{w_2 x_2 + w_2 \delta}] \\ & - [w_1(\underline{y - v_2 x_2}) + \underline{w_2 x_2}] \\ = & -w_1 v_2 \delta + w_2 \delta \\ = & \delta(-w_1 v_2 + w_2) \leq 0 \end{aligned}$$

$$w_1 \geq w_2/v_2$$



4.3 贪心法得不到最优解的处理

判别条件

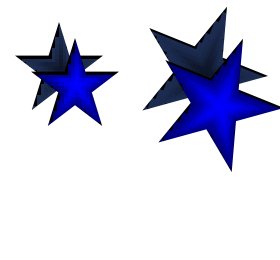
定理 对每个正整数 k , 假设对所有非负整数 y 有 $G_k(y) = F_k(y)$, 且存在 p 和 δ 满足

$$v_{k+1} = pv_k - \delta,$$

其中 $0 \leq \delta < v_k$, $v_k \leq v_{k+1}$, p 为正整数,

则下面的命题等价:

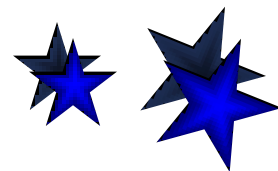
- (1) $G_{k+1}(y) = F_{k+1}(y)$ 对一切正整数 y ;
- (2) $G_{k+1}(pv_k) = F_{k+1}(pv_k)$;
- (3) $w_{k+1} + G_k(\delta) \leq pw_k$.



4.3 贪心法得不到最优解的处理

几点说明

- 根据条件(1)与(3)的等价性, 可以对 $k = 3, 4, \dots, n$, 依次利用条件(3)对贪心法是否得到最优解做出判别.
- 条件(3)验证 1 次需 $O(k)$ 时间, $k = O(n)$, 整个验证时间 $O(n^2)$
- 条件(2)是条件(1) 在 $y = pv_k$ 时的特殊情况. 若条件(1)成立, 显然有条件(2)成立. 反之, 若条件(2)不成立, 则条件(1)不成立, 钱数 $y = pv_k$ 恰好提供了一个贪心法不正确的反例.



4.3 贪心法得不到最优解的处理

验证实例

$$v_{k+1} = pv_k - \delta, \quad 0 \leq \delta < v_k, \quad p \in \mathbb{Z}^+ \\ w_{k+1} + G_k(\delta) \leq pw_k$$

例 $v_1=1, v_2=5, v_3=14, v_4=18, w_i=1,$
 $i=1,2,3,4$. 对一切 y 有

$$G_1(y)=F_1(y), \quad G_2(y)=F_2(y).$$

验证 $G_3(y) = F_3(y)$

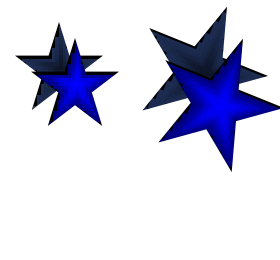
$$v_3 = pv_2 - \delta \Rightarrow p = 3, \delta = 1.$$

$$w_3 + G_2(\delta) = 1 + 1 = 2$$

$$pw_2 = 3 \times 1 = 3$$

$$w_3 + G_2(\delta) \leq pw_2$$

贪心法对于 $n=3$ 的实例得到最优解



4.3 贪心法得不到最优解的处理

验证实例

$$v_{k+1} = pv_k - \delta, \quad 0 \leq \delta < v_k, \quad p \in \mathbb{Z}^+ \\ w_{k+1} + G_k(\delta) \leq pw_k$$

例 $v_1=1, v_2=5, v_3=14, v_4=18, w_i=1,$

$i=1,2,3,4.$ 对一切 y 有

$$G_1(y)=F_1(y), G_2(y)=F_2(y), G_3(y)=F_3(y)$$

验证 $G_4(y) = F_4(y)$

$$v_4 = pv_3 - \delta \Rightarrow p=2, \delta=10$$

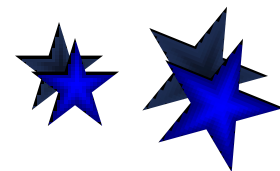
$$w_4 + G_3(\delta) = 1 + 2 = 3$$

$$pw_3 = 2 \times 1 = 2$$

$$w_4 + G_3(\delta) > pw_3$$

$$n=4, y=pv_3=28,$$

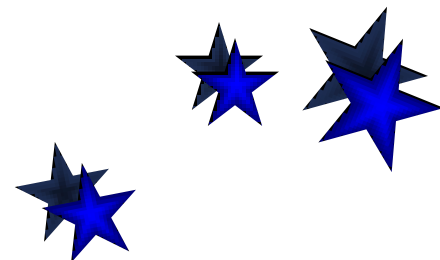
最优解: $x_3=2$, 贪心法: $x_4=1, x_2=2$



4.3 贪心法得不到最优解的处理

小结

- 贪心策略不一定得到最优解，在这种情况下可以有两种处理方法：
 - (1) 参数化分析：分析参数取什么值可得到最优解
 - (2) 估计贪心法得到的解在最坏情况下与最优解的误差
- 一个参数化分析的例子：找零钱问题



4.4 贪心法的典型应用（教材）

■ 最优前缀码——哈夫曼编码

二元前缀码

- 二元前缀码

用0-1字符串作为代码表示字符，要求任何字符的代码都不能作为其它字符代码的前缀

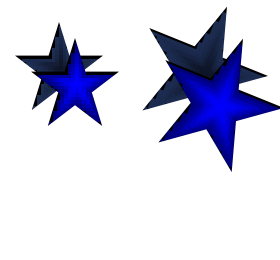
- 非前缀码的例子

$a: 001, b: 00, c: 010, d: 01$

- 解码的歧义，例如字符串 0100001

解码1: 01, 00, 001 d, b, a

解码2: 010, 00, 01 c, b, d



4.4 贪心法的典型应用

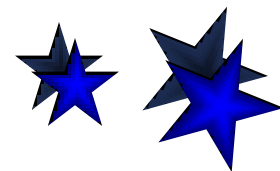
哈夫曼树算法伪码

算法 Huffman(C)

输入: $C = \{x_1, x_2, \dots, x_n\}, f(x_i), i=1, 2, \dots, n.$

输出: Q //队列

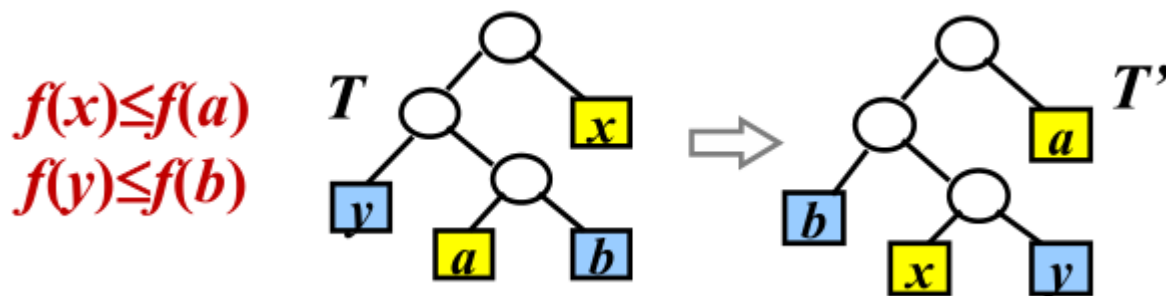
1. $n \leftarrow |C|$
2. $Q \leftarrow C$ //频率递增队列 Q
3. for $i \leftarrow 1$ to $n-1$ do
4. $z \leftarrow \text{Allocate-Node}()$ //生成结点 z
5. $z.\text{left} \leftarrow Q$ 中最小元 //最小作 z 左儿子
6. $z.\text{right} \leftarrow Q$ 中最小元 //最小作 z 右儿子
7. $f(z) \leftarrow f(x) + f(y)$
8. Insert(Q, z) // 将 z 插入 Q
9. return Q



4.4 贪心法的典型应用

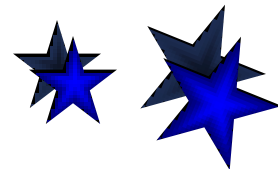
最优前缀码性质：引理1

引理1: C 是字符集, $\forall c \in C, f(c)$ 为频率, $x, y \in C$, $f(x), f(y)$ 频率最小, 那么存在最优二元前缀码使得 x, y 码字等长且仅在最后一位不同.



$$B(T) - B(T') = \sum_{i \in C} f[i]d_T(i) - \sum_{i \in C} f[i]d_{T'}(i) \geq 0$$

其中 $d_T(i)$ 为 i 在 T 中的层数 (i 到根的距离)



4.4 贪心法的典型应用

引理2

引理 设 T 是二元前缀码的二叉树, $\forall x, y \in T$, x, y 是树叶兄弟, z 是 x, y 的父亲, 令

$$T' = T - \{x, y\}$$

且令 z 的频率

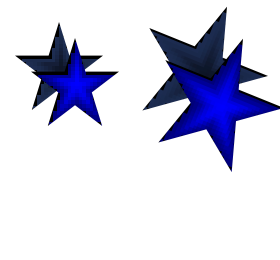
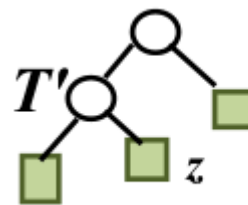
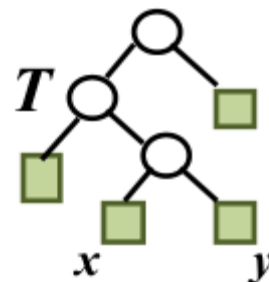
$$f(z) = f(x) + f(y)$$

T' 是对应二元前缀码

$$C' = (C - \{x, y\}) \cup \{z\}$$

的二叉树, 那么

$$B(T) = B(T') + f(x) + f(y)$$



4.4 贪心法的典型应用

引理2证明

证 $\forall c \in C - \{x, y\}$, 有

$$d_T(c) = d_{T'}(c) \Rightarrow f(c)d_T(c) = f(c)d_{T'}(c)$$

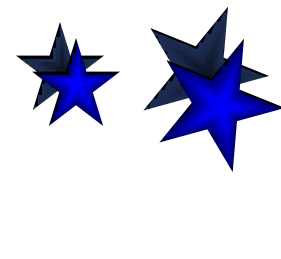
$$d_T(x) = d_T(y) = d_{T'}(z) + 1$$

$$B(T) = \sum_{i \in T} f(i)d_T(i)$$

$$= \sum_{i \in T, i \neq x, y} f(i)d_T(i) + f(x)d_T(x) + f(y)d_T(y)$$

$$= \sum_{i \in T', i \neq z} f(i)d_{T'}(i) + f(z)d_{T'}(z) + (f(x) + f(y))$$

$$= B(T') + f(x) + f(y)$$



4.4 贪心法的典型应用

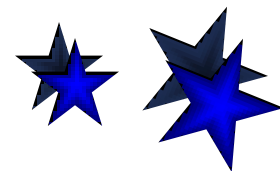
■ 哈夫曼算法的证明及应用

算法正确性证明思路

定理 Huffman 算法对任意规模为 n ($n \geq 2$) 的字符集 C 都得到关于 C 的最优前缀码的二叉树.

归纳基础 证明: 对于 $n=2$ 的字符集, Huffman算法得到最优前缀码.

归纳步骤 证明: 假设Huffman算法对于规模为 k 的字符集都得到最优前缀码, 那么对于规模为 $k+1$ 的字符集也得到最优前缀码.

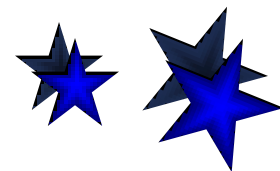
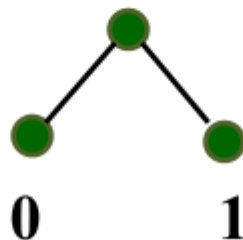


4.4 贪心法的典型应用

归纳基础

$n=2$, 字符集 $C=\{x_1, x_2\}$,

对任何代码的字符至少都需要1位二进制数字. Huffman算法得到的代码是 0 和 1, 是最优前缀码.



4.4 贪心法的典型应用

归纳步骤

假设Huffman算法对于规模为 k 的字符集都得到最优前缀码. 考虑规模为 $k+1$ 的字符集

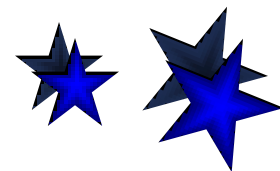
$$C = \{x_1, x_2, \dots, x_{k+1}\},$$

其中 $x_1, x_2 \in C$ 是频率最小的两个字符.

令

$$C' = (C - \{x_1, x_2\}) \cup \{z\},$$
$$f(z) = f(x_1) + f(x_2)$$

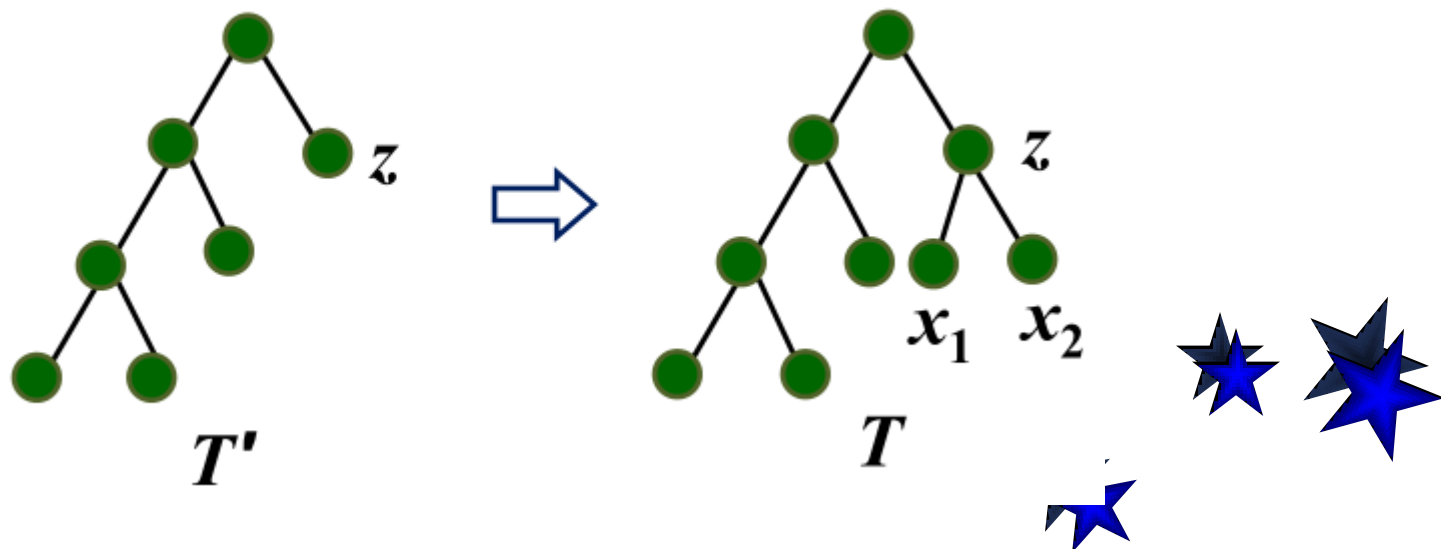
根据归纳假设, 算法得到一棵关于字符集 C' , 频率 $f(z)$ 和 $f(x_i)$ ($i=3, 4, \dots, k+1$) 的最优前缀码的二叉树 T' .



4.4 贪心法的典型应用

归纳步骤(续)

把 x_1, x_2 作为 z 的儿子附到 T' 上, 得到树 T , 那么 T 是关于 $C=(C'-\{z\})\cup\{x_1, x_2\}$ 的最优前缀码的二叉树.



4.4 贪心法的典型应用

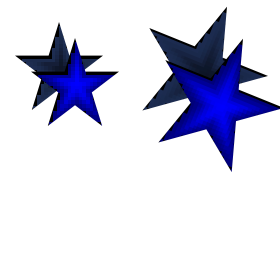
归纳步骤(续)

如若不然, 存在更优树 T^* , $B(T^*) < B(T)$,
且由引理1, 其树叶兄弟是 x_1 和 x_2 .

去掉 T^* 中 x_1 和 x_2 , 得到 $T^{*'}.$ 根据引理2

$$\begin{aligned} B(T^{*'}) &= B(T^*) - \underline{(f(x_1) + f(x_2))} \\ &< B(T) - \underline{(f(x_1) + f(x_2))} \\ &= B(T') \end{aligned}$$

与 T' 是一棵关于 C' 的最优前缀码的二叉树矛盾.



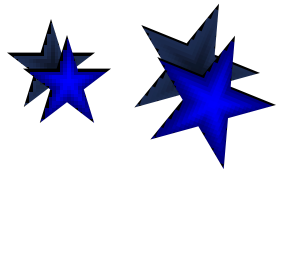
4.4 贪心法的典型应用

应用: 文件归并

问题: 给定一组不同长度的排好序文件构成的集合 $S = \{f_1, \dots, f_n\}$, 其中 f_i 表示第 i 个文件含有的项数. 使用二分归并将这些文件归并成一个有序文件.

归并过程对应于二叉树:

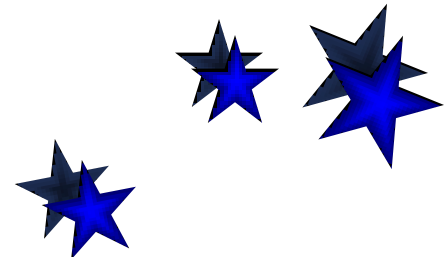
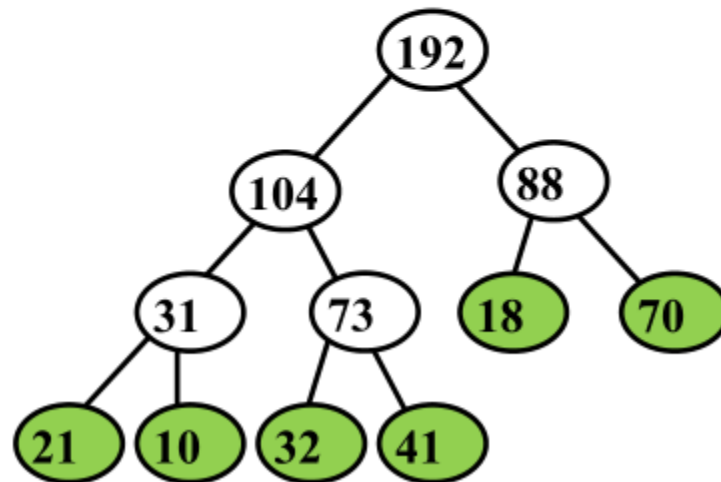
文件为树叶. f_i 与 f_j 归并的文件是它们的父结点.



4.4 贪心法的典型应用

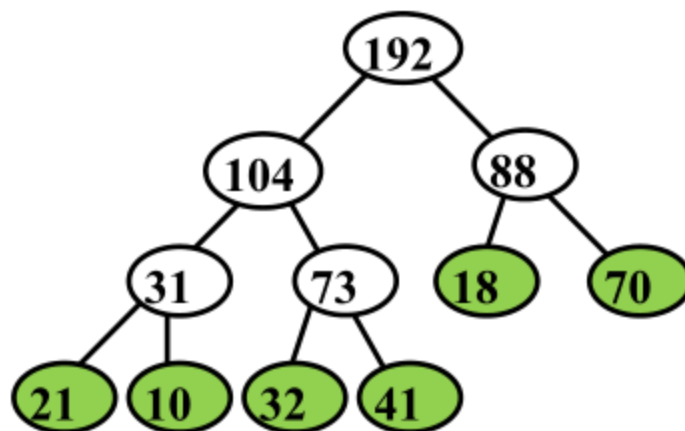
两两顺序归并

实例： $S = \{ 21, 10, 32, 41, 18, 70 \}$



4.4 贪心法的典型应用

归并代价



$$(1) (21+10-1)+(32+41-1)+(18+70-1)+ \\ (31+73-1)+(104+88-1) = 483$$

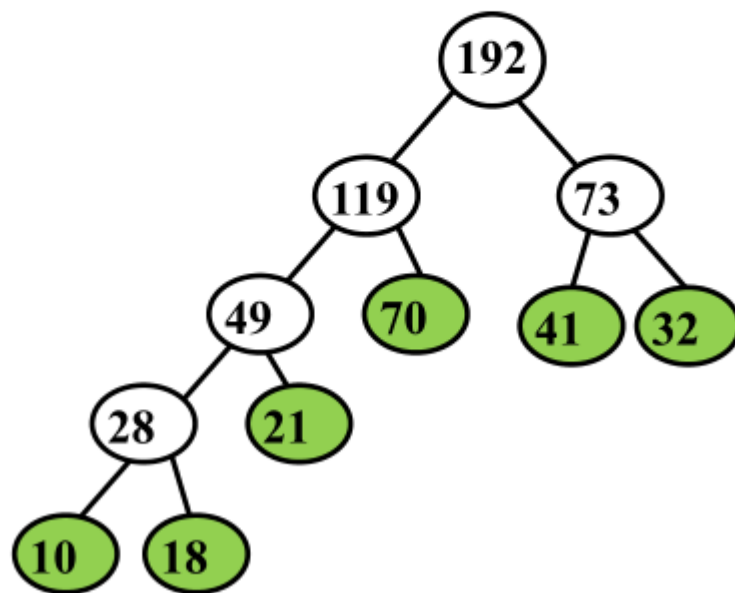
$$(2) (21+10+32+41) \times 3 + (18+70) \times 2 - 5 = 483$$

代价计算公式 $\sum_{i \in S} d(i) f_i - (n-1)$

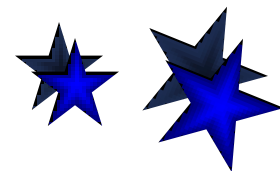
4.4 贪心法的典型应用

实例：Huffman树归并

输入： $S=\{21,10,32,41,18,70\}$



代价： $(10+18) \times 4 + 21 \times 3 + (70+41+32) \times 2 - 5 = 456$



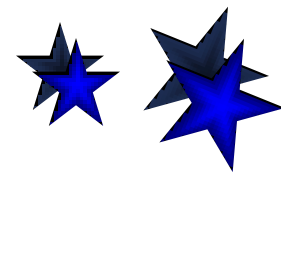
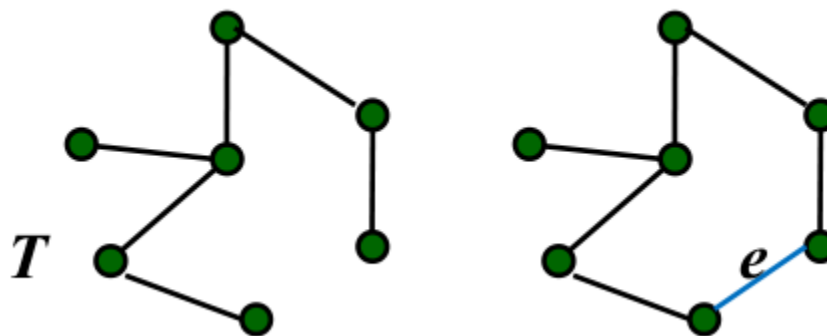
4.4 贪心法的典型应用

■ 最小生成树

生成树的性质

命题1 设 G 是 n 阶连通图, 那么

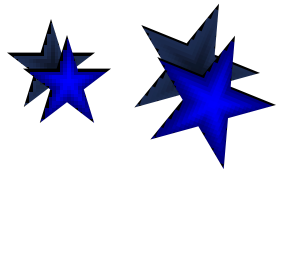
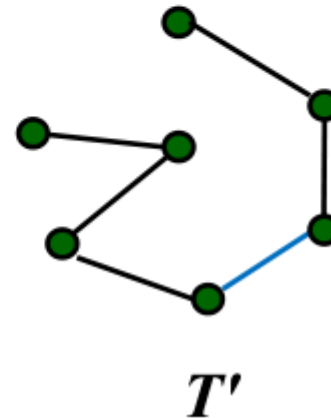
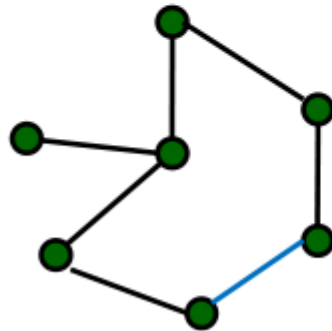
- (1) T 是 G 的生成树当且仅当 T 无圈且有 $n-1$ 条边.
- (2) 如果 T 是 G 的生成树, $e \notin T$, 那么 $T \cup \{e\}$ 含有一个圈 C (回路).



4.4 贪心法的典型应用

生成树的性质（续）

- (3) 去掉圈 C 的任意一条边，就得到 G 的另外一棵生成树 T' 。

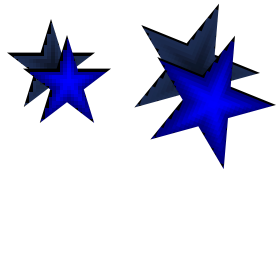


4.4 贪心法的典型应用

生成树性质的应用

- 算法步骤：选择边。
约束条件：不形成回路
截止条件：边数达到 $n-1$.
- 改进生成树 T 的方法
在 T 中加一条非树边 e , 形成回路
 C , 在 C 中去掉一条树边 e_i , 形成
一棵新的生成树 T'
$$W(T') - W(T) = W(e) - W(e_i)$$

若 $W(e) \leq W(e_i)$, 则 $W(T') \leq W(T)$



4.4 贪心法的典型应用

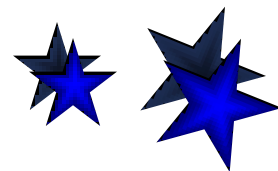
■ Prim算法

正确性证明: 归纳法

命题: 对于任意 $k < n$, 存在一棵最小生成树包含算法前 k 步选择的边.

归纳基础: $k = 1$, 存在一棵最小生成树 T 包含边 $e = \{1, i\}$, 其中 $\{1, i\}$ 是所有关联 1 的边中权最小的.

归纳步骤: 假设算法前 k 步选择的边构成一棵最小生成树的边, 则算法前 $k+1$ 步选择的边也构成一棵最小生成树的边.

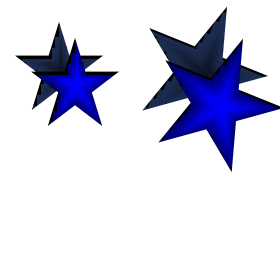
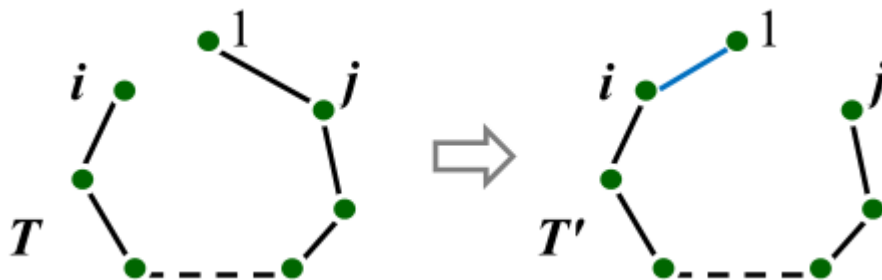


4.4 贪心法的典型应用

归纳基础

证明： 存在一棵最小生成树 T 包含关联结点1的最小权的边 $e=\{1,i\}$.

证 设 T 为一棵最小生成树，假设 T 不包含 $\{1,i\}$ ，则 $T \cup \{\{1,i\}\}$ 含有一条回路，回路中关联1的另一条边 $\{1,j\}$. 用 $\{1,i\}$ 替换 $\{1,j\}$ 得到树 T' ，则 T' 也是生成树，且 $W(T') \leq W(T)$.

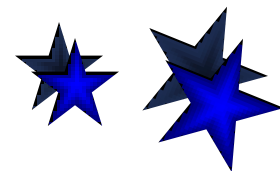
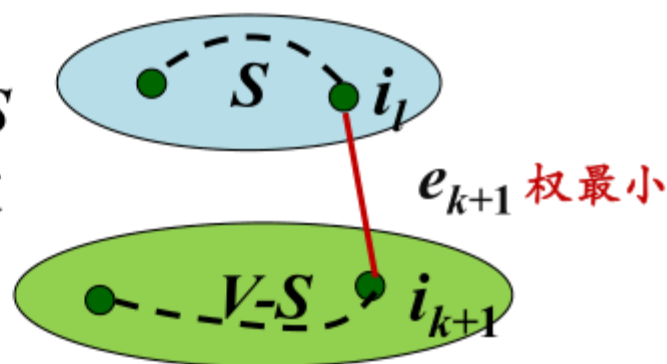


4.4 贪心法的典型应用

归纳步骤

假设算法进行了 k 步, 生成树的边为 $e_1, e_2, \dots, e_k$, 这些边的端点构成集合 S . 由归纳假设存在 G 的一棵最小生成树 T 包含这些边.

算法第 $k+1$ 步选择顶点 i_{k+1} , 则 i_{k+1} 到 S 中顶点边权最小, 设此边 $e_{k+1} = \{i_{k+1}, i_l\}$. 若 $e_{k+1} \in T$, 算法 $k+1$ 步显然正确.



4.4 贪心法的典型应用

归纳步骤（续）

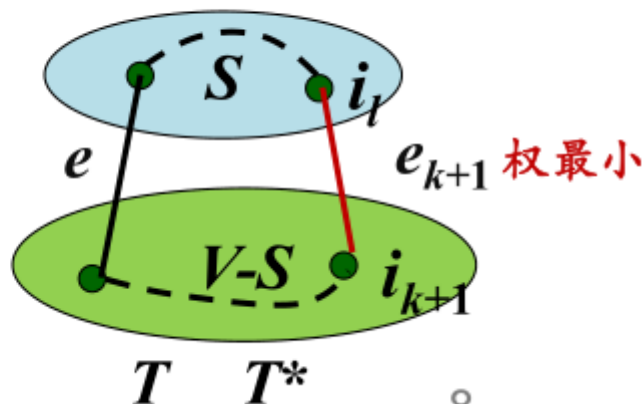
假设 T 不含有 e_{k+1} ，则将 e_{k+1} 加到 T 中形成一条回路。这条回路有另外一条连接 S 与 $V-S$ 中顶点的边 e ，

令 $T^* = (T - \{e\}) \cup \{e_{k+1}\}$

则 T^* 是 G 的一棵生成树，包含 $e_1, e_2, \dots, e_{k+1}$ ，且

$$W(T^*) \leq W(T)$$

算法到 $k+1$ 步仍得到最小生成树。



4.4 贪心法的典型应用

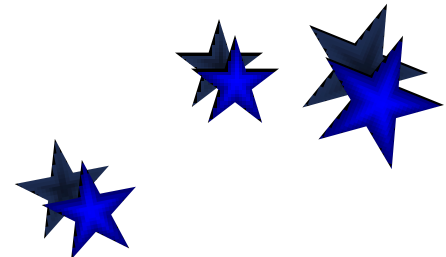
时间复杂度

算法步骤执行 $O(n)$ 次

每次执行 $O(n)$ 时间：

找连接 S 与 $V-S$ 的最短边

算法时间： $T(n) = O(n^2)$



4.4 贪心法的典型应用

■ Kruskal 算法

正确性证明思路

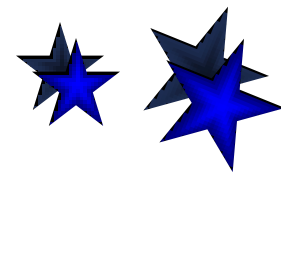
命题：对于任意 n , 算法对 n 阶图找到一棵最小生成树.

证明思路:

归纳基础 证明: $n = 2$, 算法正确.

G 只有一条边, 最小生成树就是 G .

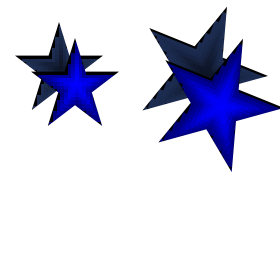
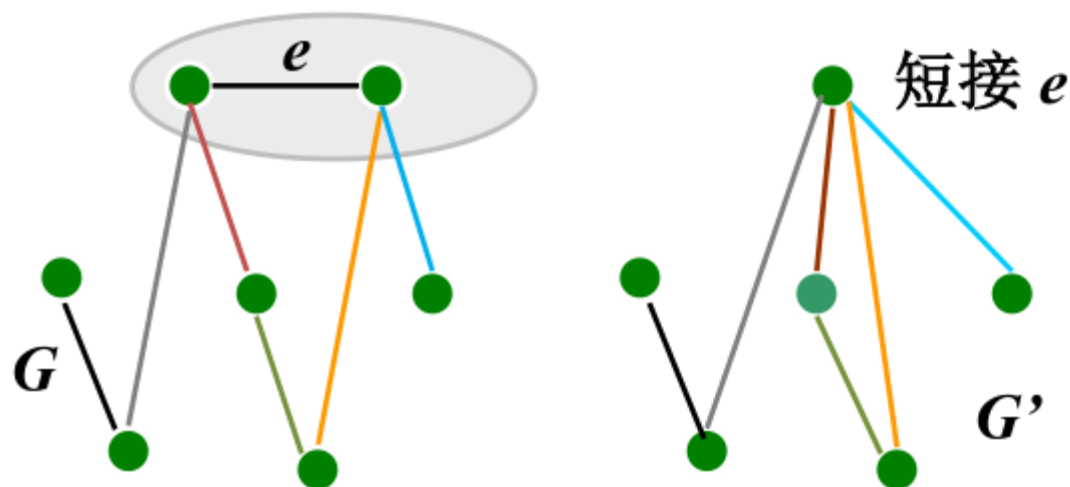
归纳步骤 证明: 假设算法对于 n 阶图是正确的, 其中 $n > 1$, 则对于任何 $n+1$ 阶图算法也得到一棵最小生成树.



4.4 贪心法的典型应用

短接操作

任给 $n+1$ 个顶点的图 G , G 中最小权边 $e = \{i, j\}$, 从 G 中短接 i 和 j , 得到图 G' .



4.4 贪心法的典型应用

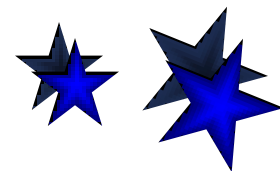
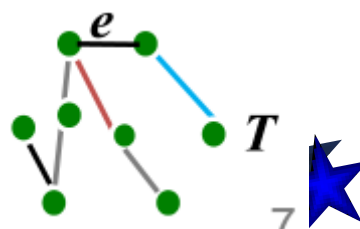
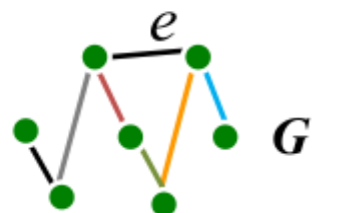
归纳步骤证明

对于任意 $n+1$ 阶图 G 短接最短边 e , 得到 n 阶图 G'

根据归纳假设算法得到 G' 的最小生成树 T'

将被短接的边 e “拉伸”回到原来长度, 得到树 T

证明 T 是 G 的最小生成树



4.4 贪心法的典型应用

T 是 G 的最小生成树

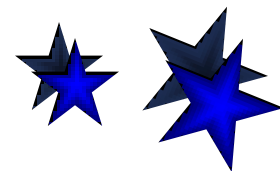
$T = T' \cup \{e\}$ 是关于 G 的最小生成树.

否则存在 G 的含边 e 的最小生成树 T^* , $W(T^*) < W(T)$. (如果 $e \notin T^*$, 在 T^* 中加边 e , 形成回路. 去掉回路中任意的边所得生成树的权仍旧最小).

在 T^* 短接 e 得到 G' 的生成树 $T^* - \{e\}$,

$$\begin{aligned} W(T^* - \{e\}) &= W(T^*) - w(e) \\ &< W(T) - w(e) = W(T') \end{aligned}$$

与 T' 的最优性矛盾.



4.4 贪心法的典型应用

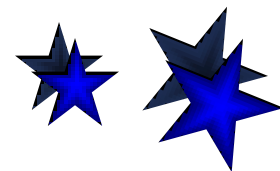
算法实现与时间复杂度

建立FIND数组， $\text{FIND}[i]$ 是结点 i 的连通分支标记。

- (1) 初始 $\text{FIND}[i] = i$.
- (2) 连通分支合并，较小分支标记更新为较大分支标记

每个结点至多更新 $\log n$ 次，
建立和更新 FIND 数组: $O(n \log n)$

时间: $O(m \log m) + O(n \log n) + O(m)$
 $= O(m \log n)$



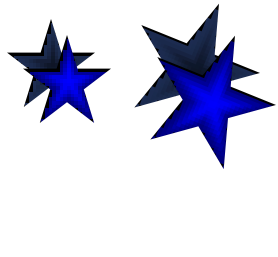
4.4 贪心法的典型应用

应用：数据分组问题

一组数据(照片, 文件, 生物标本)要把它们按照相关性进行分类.

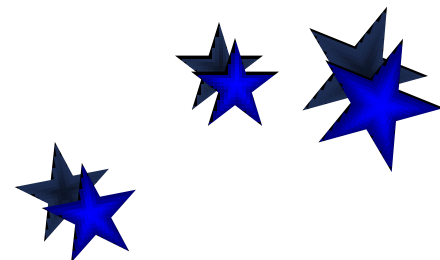
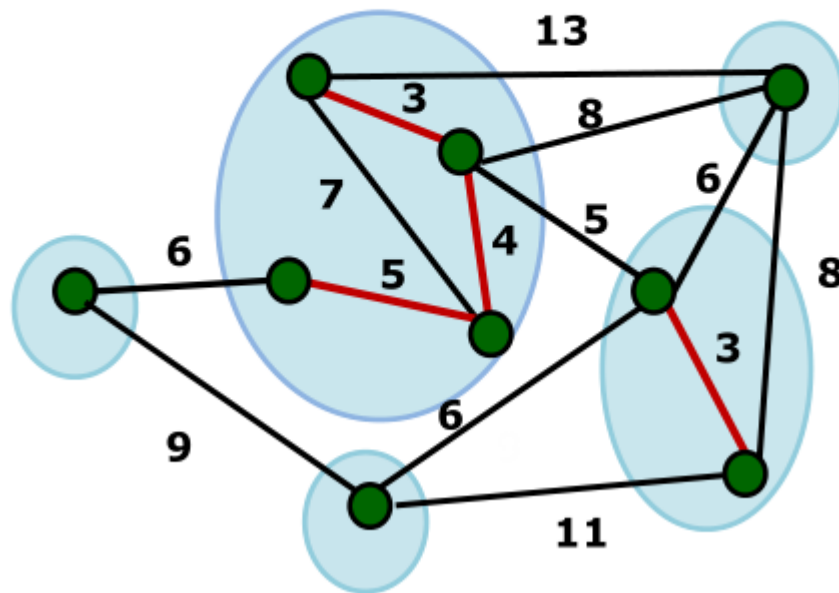
用相似度函数或“距离”来描述个体之间的差距.

如果分成5类, 使得每类内部的个体尽可能相近, 不同类之间的个体尽可能地“远离”. 如何划分?



4.4 贪心法的典型应用

应用：数据分组问题

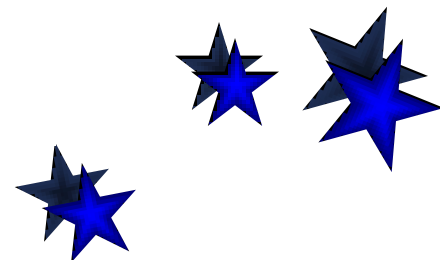


4.4 贪心法的典型应用

单链聚类

类似于Kruskal算法

- (1) 按照边长从小到大对边排序
- (2) 依次考察当前最短边 e ，如果 e 与已经选中的边不构成回路，则把 e 加入集合，否则跳过 e . 计数图的连通分支个数.
- (3) 直到保留了 k 个连通分支为止.



4.4 贪心法的典型应用

- 单源最短路径：Dijkstra算法

归纳证明思路

命题：当算法进行到第 k 步时，对于 S 中每个结点 i ,

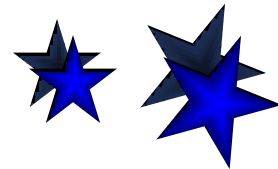
$$dist[i] = short[i]$$

归纳基础

$$k = 1, S = \{s\}, dist[s] = short[s] = 0.$$

归纳步骤

证明：假设命题对 k 为真，则对 $k+1$ 命题也为真。



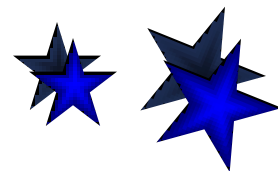
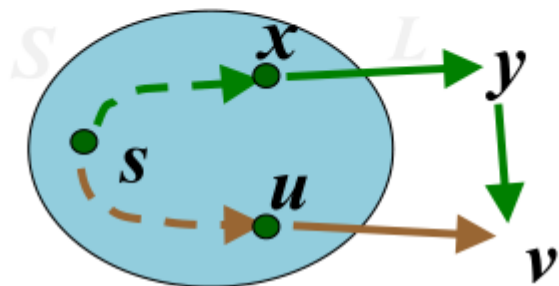
4.4 贪心法的典型应用

归纳步骤证明

假设命题对 k 为真, 考虑 $k+1$ 步算法
选择顶点 v (边 $\langle u, v \rangle$). 需要证明

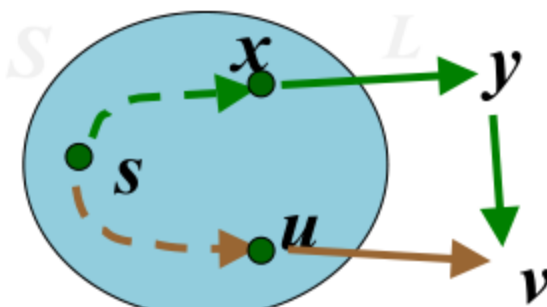
$$\text{dist}[v] = \text{short}[v]$$

若存在另一条 s - v 路径 L (绿色), 最后一次出 S 的顶点为 x , 经过 $V-S$ 的第一个顶点 y , 再由 y 经过一段在 $V-S$ 中的路径到达 v .



4.4 贪心法的典型应用

归纳步骤证明（续）



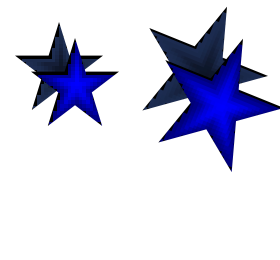
在 $k+1$ 步算法选择顶点 v ，而不是 y ，

$$\text{dist}[v] \leq \text{dist}[y]$$

令 y 到 v 的路径长度为 $d(y, v)$

$$\text{dist}[y] + d(y, v) \leq L$$

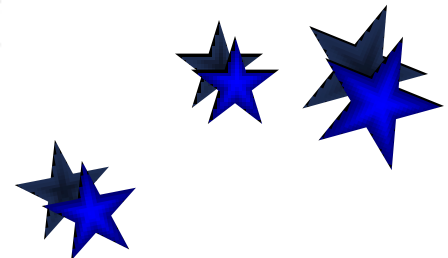
于是 $\text{dist}[v] \leq L$ ，即 $\text{dist}[v] = \text{short}[v]$



4.4 贪心法的典型应用

时间复杂度

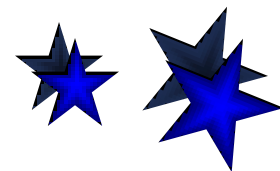
- 时间复杂度: $O(nm)$
算法进行 $n-1$ 步
每步挑选1个具有最小 $dist$ 函数值的
结点进入 S , 需要 $O(m)$ 时间
- 选用基于堆实现的优先队列的数据结构, 可以将算法时间复杂度
降到 $O(m \log n)$



4.4 贪心法的典型应用

贪心法小结

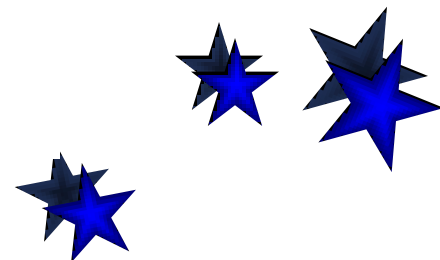
- 贪心法适用于组合优化问题.
- 求解过程是多步判断过程, 最终的判断序列对应于问题的最优解.
- 判断依据某种“短视的”贪心选择性质, 性质的好坏决定了算法的正确性. 贪心性质的选择往往依赖于直觉或者经验.



4.4 贪心法的典型应用

贪心法小结（续）

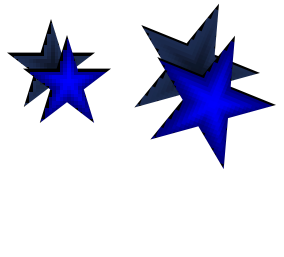
- 贪心法正确性证明方法：
 - (1) 直接计算优化函数，贪心法的解恰好取得最优值
 - (2) 数学归纳法（对算法步数或者问题规模归纳）
 - (3) 交换论证
- 证明贪心策略不对：举反例



4.4 贪心法的典型应用

贪心法小结（续）

- 对于某些不能保证对所有的实例都得到最优解的贪心算法（近似算法），可做参数化分析或者误差分析.
- 贪心法的优势：算法简单，时间和空间复杂性低



4.4 贪心法的典型应用

贪心法小结（续）

- 几个著名的贪心算法
最小生成树的Prim算法
最小生成树的Kruskal算法
单源最短路的Dijkstra算法

